AI WEATHER WISE- DELIVER INTELLIGENT WEATHER FORECASTS AND INSIGHTS

INTRODUCTION :

Project title : Weather Wise

Team ID: SWTID-2026-7040

Team leader :

Dharshini. V (dharshk2824@gmail.com)

Team members:

R.Dharshana (dharshanark28@gmail.com)

K.DivyaPriya(divyakumerval@gmail.com)

S.Durga(durgasakthivel6374@gmail.com)

DESCRIPTION

Weather Wise is a weather forecasting and information system designed to provide users with accurate, simple, and easy-to-understand weather information. The project helps users check current weather conditions, temperature, humidity, wind speed, rainfall probability, and weather forecasts for selected locations.

The main purpose of Weather Wise is to help users make better day-to-day decisions based on weather conditions. Users can search for a city or location and view its current weather details along with upcoming forecasts. The system can also provide weather alerts for conditions such as heavy rain, storms, extreme temperatures, or strong winds.

SCENARIO-BASED CASE STUDY

BACKGROUND

Consider a situation where a student is planning to attend college and participate in an outdoor event. Before leaving home, the student wants to know whether there is a possibility of heavy rain or strong winds during the day. By using Weather Wise, the student can search for the required location and view the current weather conditions and forecast.

Similarly, farmers can use weather information to plan irrigation and agricultural activities, travelers can check weather conditions before starting a journey, and event organizers can monitor weather conditions before conducting outdoor programs.

PROBLEM

- Difficulty understanding complex weather data.
- Sudden weather changes affecting daily activities.
- Difficulty planning outdoor activities based on weather conditions.

- Lack of timely weather alerts for potentially severe conditions.
- Need for weather information in a simple and user-friendly format.

SOLUTION

The Weather Wise project provides a simple and user-friendly solution for accessing weather information and forecasts in one place. The system allows users to enter or select a location and view the relevant weather conditions.

USAGE

The **Weather Wise** system can be used by different types of users to understand weather conditions and plan their activities accordingly.

- **User Usage:** The **Weather Wise** system is designed to be simple and easy for users to access weather information
- **Developer/Admin Usage:** The Developer/Admin Page is used to manage and monitor the Weather Wise application **.** .
- **Weather Application User:** The User Module of the Weather Wise application allows users to easily access and understand weather information for their required location.

OUTCOME

The Weather Wise project provides a simple and user-friendly platform for accessing and understanding weather information. The system helps users obtain weather details for different locations and use forecasts for better planning.

1. Software Requirements

1. Operating System: Windows 10 / Windows 11 Android (for mobile application, if applicable)

2. Node.js : Server-side runtime environment

3. Npm: Node package manager to control operational dependencies.

4. Gemini AI SDK: Integration using @google/generative-ai for natural language insights

5. MongoDB: Document NoSQL engine storing users, posts, categories,
commands, and analytics metrics.

6. Postman: API verification toolkit to assert schema validation response
across administrative route shields.

7. Code Editor: Visual studio code or similar IDE Environment

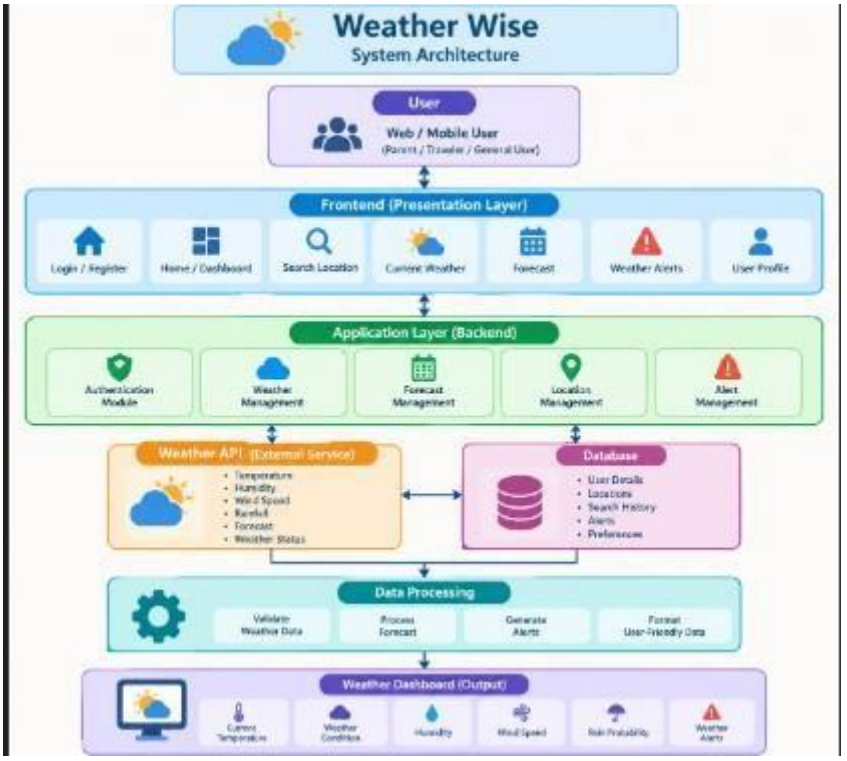
2 . Hardware Requirements

1. Processor: Intel Core i5 (8th Gen or above)/AMD Ryzen 5 or equivalent

2. RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, Processing layers, and execution nodes).

2 . Storage: 1 GB of available disk workspace.

Epic 1: PROJECT ARCHITECTURE



The AI Weather Wise application follows the MVC pattern to decouple business logic from data management. The Controller layer orchestrates weather fetching from OpenweatherMap and generation of AI insights via Gemini, While the Model layer manages user and location entities.

Components Definition

Weather Wise is a weather information and forecasting application that provides users with accurate and useful weather-related information for a selected location.

1. User Interface: The user interface provides a simple and user-friendly screen for users to search locations and view current weather, forecasts, and weather alerts.

2 . Express Server Gateway: The Express Server Gateway handles communication between the Weather Wise user interface and the weather API, processes requests, and sends the required weather data back to the application.

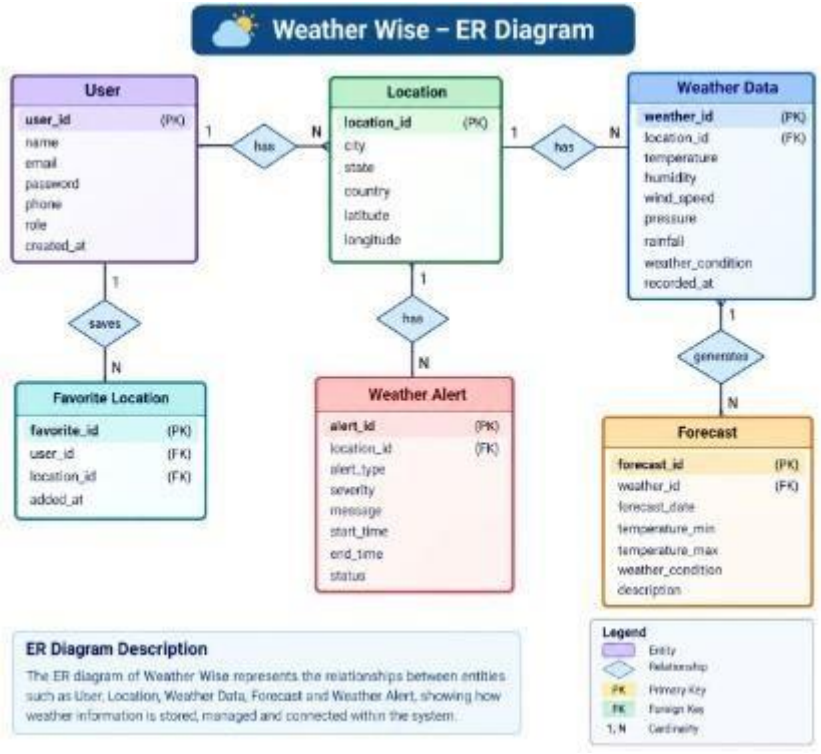
3 . Authentication: Authentication securely verifies users through login and registration, ensuring that only authorized users can access Weather Wise features and personal information.

4 . Granular Services: Granular services divide the Weather Wise application into small, independent services that handle specific functions such as weather data, forecasts, location search, and alerts.

5 . Database Interface Layer: The Database Interface Layer connects the Weather Wise application with the database to securely store, retrieve, update, and manage user and weather-related information.

6. AI Service Integration: AI Service Integration uses artificial intelligence to analyze weather data and provide personalized weather predictions, recommendations, and alerts to users.

Entity-Relationship (ER) Diagram Description



The data architecture manages User profiles and their associated favorite Locations through logical references:

1 . User Entity

- _id (Primary Key - ObjectId)

- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "reader")

2. Location Entity

- _id (Primary Key - ObjectId)
- user (Foreign Key referencing User Collection)
- city (String, Required)
- country (String, Required)
- createdAt (Date, Default: Now)

Key Features

- Real-time weather updates
- Location-based weather information
- Current temperature and weather conditions
- Hourly and daily weather forecasts
- Humidity and wind speed details
- Rainfall information
- Weather alerts and notifications
- Favorite location management
- Weather search by city
- AI-based weather recommendations

Roles and Responsibilities

1. User

- Register and log in to the Weather Wise application. ● Search and view weather information.
- Check current and future weather forecasts.
- Save and manage favorite locations.
- Receive weather alerts and notifications.

2. Administrator

- Manage registered users.
- Manage weather-related data and application settings. ● Monitor system performance and activities.
- Manage weather alerts and notifications.
- Maintain database security and system reliability.

User Flow

1 . User logs in or registers.

2 . User saves a favorite city via the API.

3 . User fetches current weather metrics for that city.

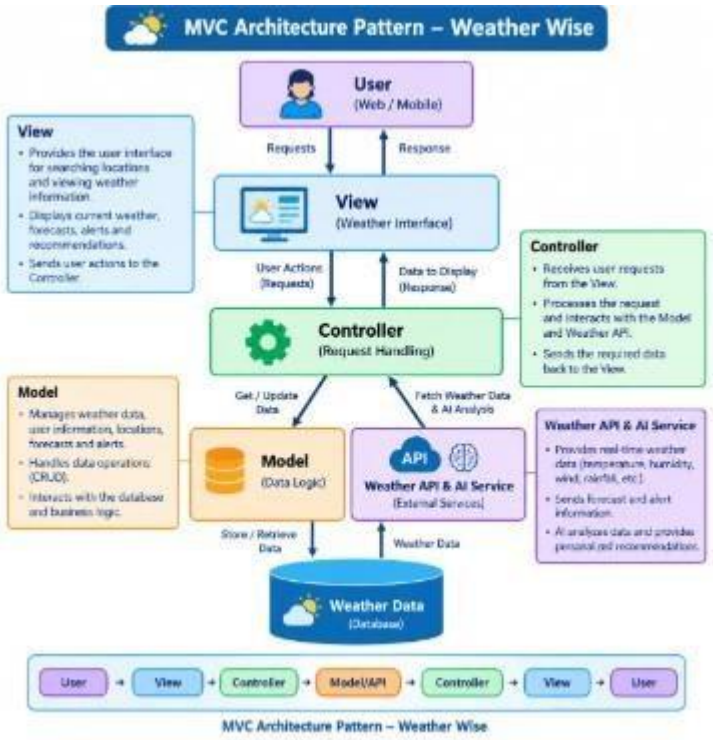
4 . User requests AI-driven insights (summaries/recommendations) based on the weather data.

5 . System validates JWT and returns the requested information.



MVC Architecture Pattern

III The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



Model Layer (Data Blueprint)

Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.

View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API **Routing Layer** . It links network requests on specified paths directly to their associated controller handler logic maps.

Epic 2: Project Setup and Configuration

Creating the Project Folder

- Create a new folder named **AI Weatherwise** ● Open the project folder in **Visual Studio Code**.
- Initialize the project using Node.js and install the required dependencies

Project Initialization

Open the project folder in the VS Code terminal and execute:

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

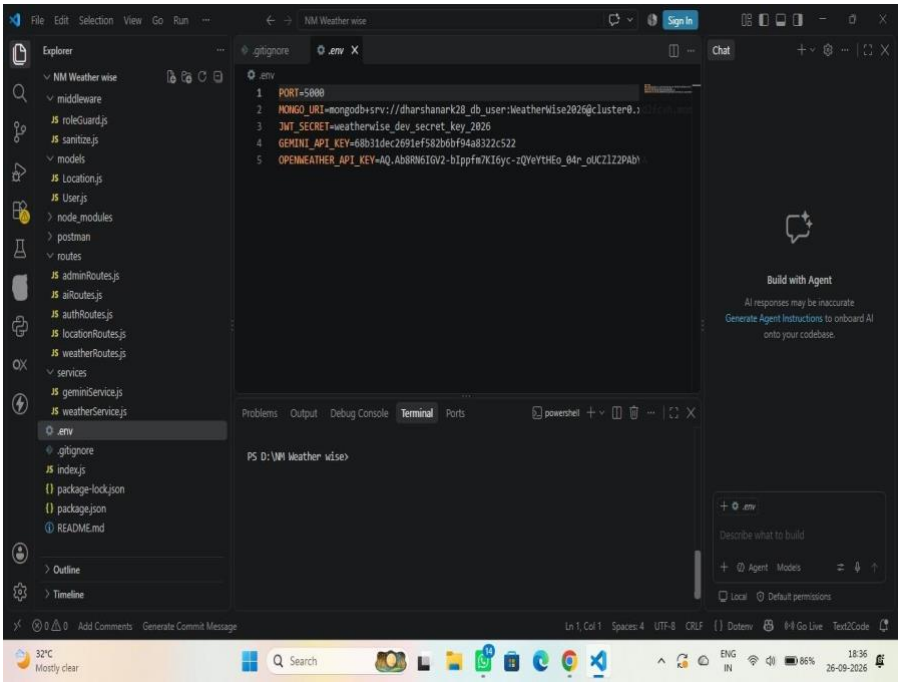
Install Nodemon for development:

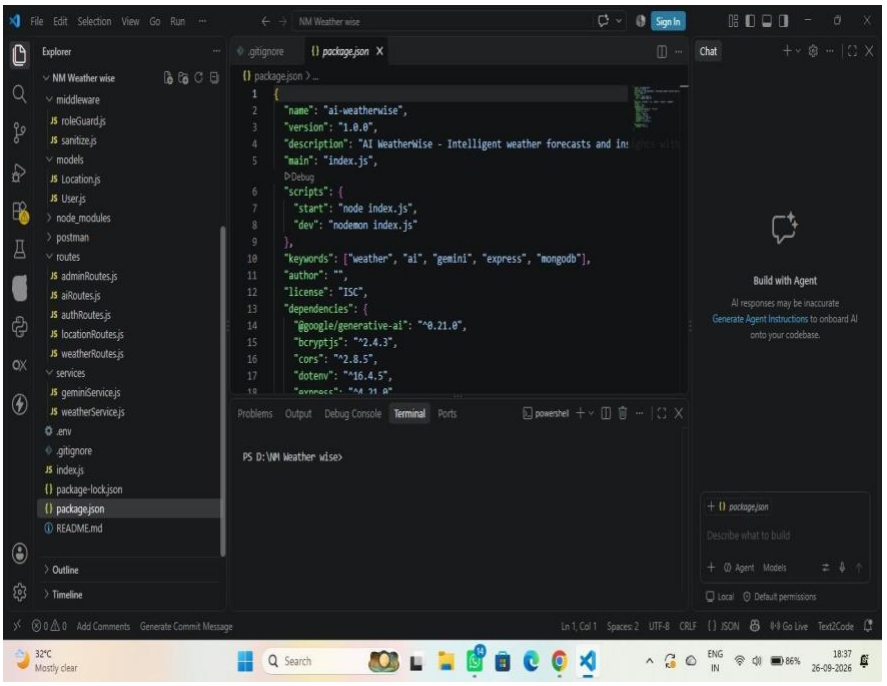
```
npm install --save-dev nodemon
```

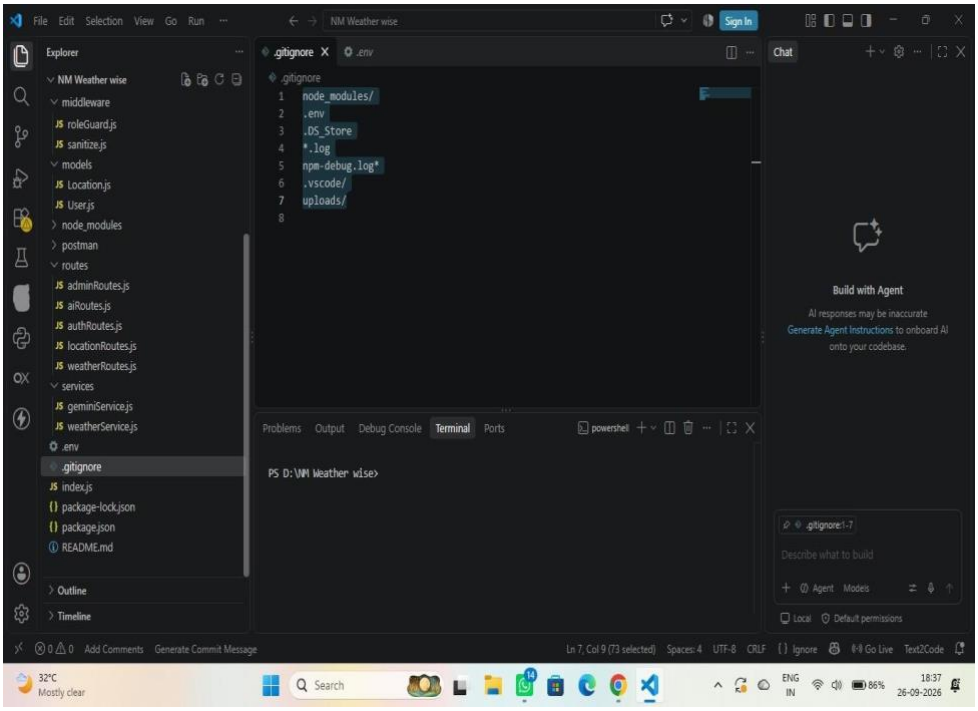
Create the Following Files

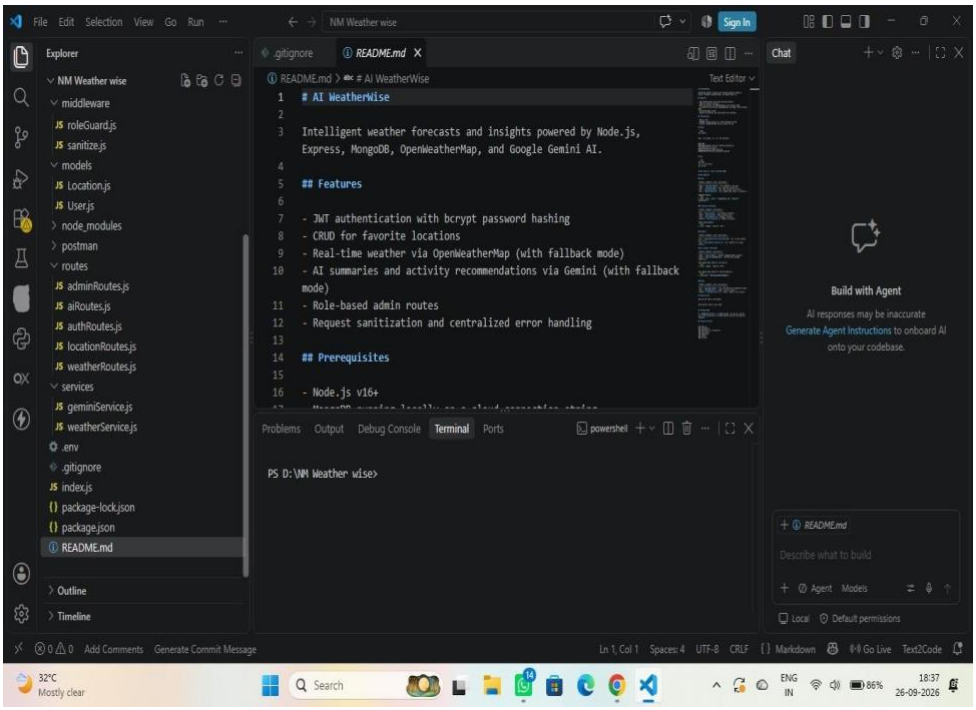
- index.js
- .env
- package.json
- .gitignore

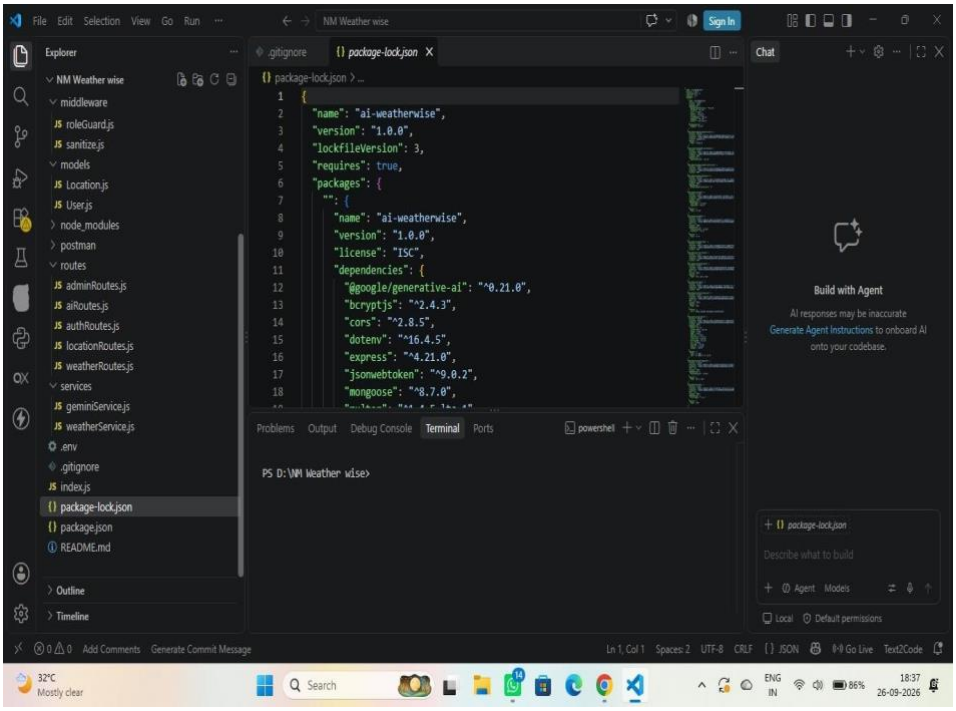
CODING :



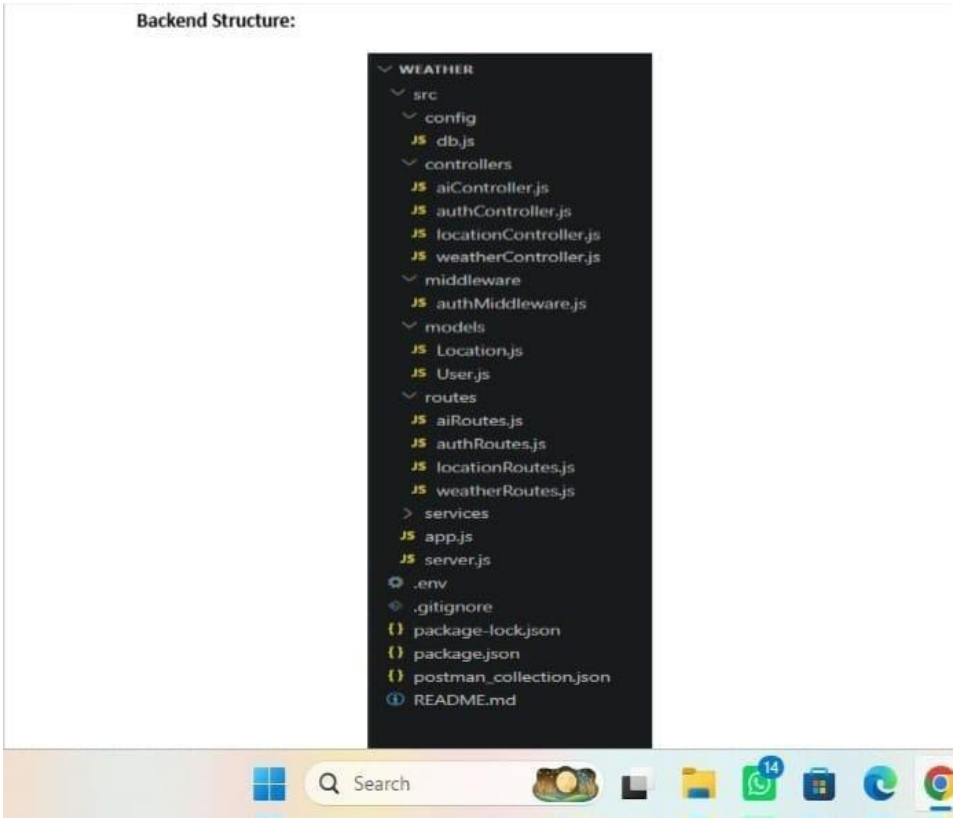

File Edit Selection View Go Run
NM Weather wise
Sign In
Explorer
NM Weather wise
middleware
roleGuard.js
sanitize.js
models
Location.js
User.js
node_modules
postman
routes
adminRoutes.js
aiRoutes.js
authRoutes.js
locationRoutes.js
weatherRoutes.js
services
geminiService.js
weatherService.js
.env
.gitignore
index.js
package-lock.json
package.json
README.md
Outline
Timeline
.gitignore
package.json
package.json > ...
{
"name": "ai-weatherwise",
"version": "1.0.0",
"description": "AI WeatherWise - Intelligent weather forecasts and ins
"main": "index.js",
Debug
"scripts": {
"start": "node index.js",
"dev": "nodemon index.js"
},
"keywords": ["weather", "ai", "gemini", "express", "mongodb"],
"author": "",
"license": "ISC",
"dependencies": {
"@google/generative-ai": "^0.21.0",
"bcryptjs": "^2.4.3",
"cors": "^2.8.5",
"dotenv": "^16.4.5",
Problems Output Debug Console Terminal Ports
powershell
PS D:\NM Weather wise>
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
package.json
Describe what to build
Agent Models
Local Default permissions
0 0 Add Comments Generate Commit Message
Ln 1, Col 1 Spaces: 2 UTF-8 CRLF { } JSON Go Live Text2Code
32°C
Mostly clear
Search
ENG
IN
86%
18:37
26-09-2026


File Edit Selection View Go Run
NM Weather wise
Sign In
Explorer
NM Weather wise
middleware
roleGuard.js
sanitize.js
models
Location.js
User.js
node_modules
postman
routes
adminRoutes.js
aiRoutes.js
authRoutes.js
locationRoutes.js
weatherRoutes.js
services
geminiService.js
weatherService.js
.env
.gitignore
index.js
package-lock.json
package.json
README.md
Outline
Timeline
.gitignore
.env
1 node_modules/
2 .env
3 .DS_Store
4 *.log
5 npm-debug.log*
6 .vscode/
7 uploads/
8
Problems Output Debug Console Terminal Ports
powershell
PS D:\NM Weather wise>
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
.gitignore:1-7
Describe what to build
Agent Models
Local Default permissions
0 0 Add Comments Generate Commit Message
Ln 7, Col 9 (73 selected) Spaces: 4 UTF-8 CRLF Ignore Go Live Text2Code
32°C
Mostly clear
Search
ENG
IN
86%
18:37
26-09-2026

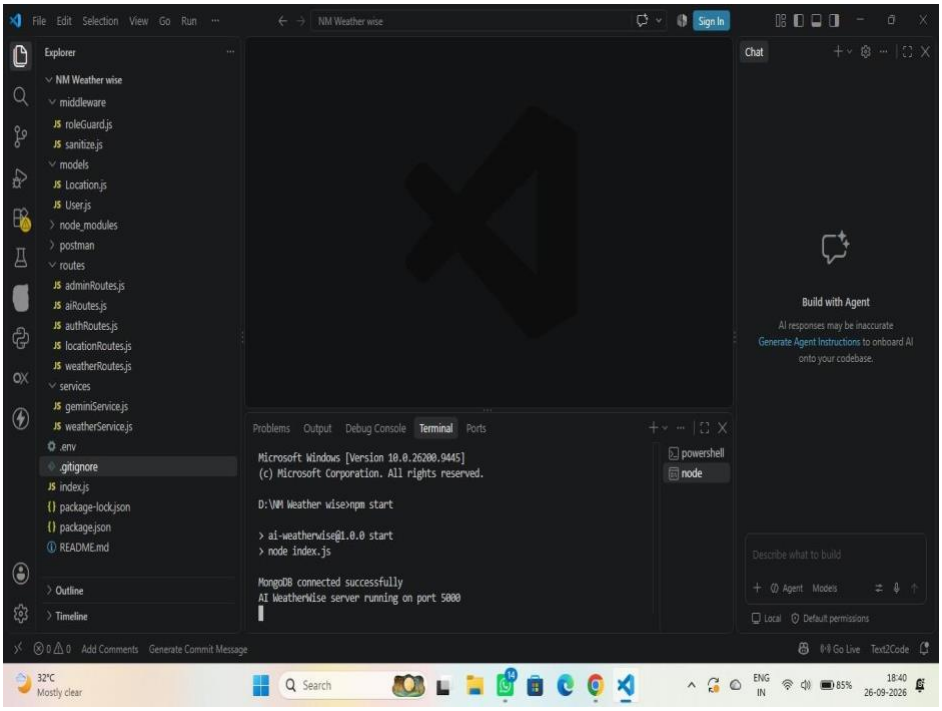

File Edit Selection View Go Run
NM Weather wise
Sign In
Explorer
NM Weather wise
middleware
roleGuard.js
sanitize.js
models
Location.js
User.js
node_modules
postman
routes
adminRoutes.js
aiRoutes.js
authRoutes.js
locationRoutes.js
weatherRoutes.js
services
geminiService.js
weatherService.js
.env
.gitignore
index.js
package-lock.json
package.json
README.md
Outline
Timeline
.gitignore
README.md
README.md > # AI WeatherWise
Text Editor
1 # AI WeatherWise
2
3 Intelligent weather forecasts and insights powered by Node.js,
Express, MongoDB, OpenWeatherMap, and Google Gemini AI.
4
5 ## Features
6
7 - JWT authentication with bcrypt password hashing
8 - CRUD for favorite locations
9 - Real-time weather via OpenWeatherMap (with fallback mode)
10 - AI summaries and activity recommendations via Gemini (with fallback
mode)
11 - Role-based admin routes
12 - Request sanitization and centralized error handling
13
14 ## Prerequisites
15
16 - Node.js v16+
Problems Output Debug Console Terminal Ports
powershell
PS D:\NM Weather wise>
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
README.md
Describe what to build
Agent Models
Local Default permissions
0 0 Add Comments Generate Commit Message
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Markdown Go Live Text2Code
32°C
Mostly clear
Search
ENG
IN
86%
18:37
26-09-2026



Backend Structure:



OUTPUT :


File Edit Selection View Go Run
NM Weather wise
Sign In
Explorer
NM Weather wise
middleware
roleGuard.js
sanitize.js
models
Location.js
User.js
node_modules
postman
routes
adminRoutes.js
aiRoutes.js
authRoutes.js
locationRoutes.js
weatherRoutes.js
services
geminiService.js
weatherService.js
.env
.gitignore
index.js
package-lock.json
package.json
README.md
Outline
Timeline
Problems Output Debug Console Terminal Ports
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.
D:\NM Weather wise>npm start
> ai-weatherwise@1.0.0 start
> node index.js
MongoDB connected successfully
AI WeatherWise server running on port 5000
powershell
node
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Describe what to build
Agent Models
Local Default permissions
0 0 Add Comments Generate Commit Message
Go Live Text2Code
32°C
Mostly clear
Search
ENG
IN
85%
18:40
26-09-2026